Imputation Impact on Model Performance

This notebook evaluates whether imputing missing values in training data **genuinely improves** or **degrades** model performance, compared to simply training on complete-records only.

Experimental Design

Two models are trained and their performance compared on the same clean (complete-record) reference set:

	Model 1 — Complete-record	Model 2 — Imputed Full Data
Train	Ground-truth rows only; missing rows dropped	All rows; missing values imputed
Test	Complete-record reference rows	Same complete-record reference rows

Restricting the test set to complete-record rows for **both** models ensures any performance difference is attributable to training data quality — not evaluation on fabricated values.

> **Known limitation:** Model 2 trains on more rows (N_complete + N_imputed vs N_complete). A performance gain therefore cannot be attributed solely to imputation quality — more training data is a confound. This is declared upfront as a design trade-off.

> **Mitigation**: Subsampling Model 2's training data to match Model 1's size controls for the confound, at the cost of added complexity and nitty-gritty in evaluation.

In [1]:
```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, RobustScaler, TargetEncoder
from sklearn.impute import SimpleImputer
from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier

import warnings
warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', None)
```

In [2]:
```
df = pd.read_csv('../data/raw/ecommerce_customer_churn_dataset.csv')
df.head()
```

Out[2]:

	Age	Gender	Country	City	Membership_Years	Login_Frequency	Session_Duration_Avg	Pag
0	43.0	Male	France	Marseille	2.9	14.0	27.4	
1	36.0	Male	UK	Manchester	1.6	15.0	42.7	
2	45.0	Female	Canada	Vancouver	2.9	10.0	24.8	
3	56.0	Female	USA	New York	2.6	10.0	38.4	
4	35.0	Male	India	Delhi	3.1	29.0	51.4	

Model 1 — Complete-Case Training

All rows that contain **any** missing value are dropped. The model is trained exclusively on rows where every feature is known.

In [3]:
```
df_dropped = df.dropna().copy()

X = df_dropped.drop('Churned', axis=1)
y = df_dropped['Churned']

Xtrain, Xrest, ytrain, yrest = train_test_split(
    X, y, train_size=0.7, stratify=y, random_state=42
)

print('All rows with no missing values only:')
print(f'Train size: {len(Xtrain):,}  |  Test size: {len(Xrest):,}')
```

```
All rows with no missing values only:
Train size: 12,588  |  Test size: 5,395
```

Multicollinearity Check (VIF)

In [4]:
```
num_cols = X.select_dtypes(exclude='object').columns
vif_data = [
    variance_inflation_factor(X[num_cols].values, i)
    for i in range(num_cols.size)
]
vif = pd.DataFrame({'feature': num_cols, 'VIF': vif_data})
```

In [5]:
```
vif.sort_values('VIF')
```

Out[5]:

	feature	VIF
8	Average_Order_Value	1.580026
9	Days_Since_Last_Purchase	1.997038
11	Returns_Rate	2.611092
1	Membership_Years	3.053466
14	Product_Reviews_Written	4.670723
10	Discount_Usage_Rate	4.715345
17	Payment_Method_Diversity	5.277947
19	Credit_Balance	6.133862
13	Customer_Service_Calls	6.135915
6	Wishlist_Items	6.438491
15	Social_Media_Engagement_Score	6.661899
18	Lifetime_Value	6.851449
12	Email_Open_Rate	7.116230
2	Login_Frequency	8.706776
0	Age	10.001402
5	Cart_Abandonment_Rate	12.812410
7	Total_Purchases	13.650890
16	Mobile_App_Usage	16.903379
4	Pages_Per_Session	20.370394
3	Session_Duration_Avg	27.898277

`Session_Duration_Avg` and `Pages_Per_Session` sit on the higher end of the VIF range, which is expected in an e-commerce context — more pages per session leads to longer session duration and vice versa.

All columns are kept for this baseline; dropping or combining these is left as a future experiment.

Encoding Categorical Features

In [6]:
```
Xtrain.select_dtypes('O')
```

Out[6]:

	Gender	Country	City	Signup_Quarter
19443	Female	France	Toulouse	Q2
25173	Male	USA	Houston	Q3
11256	Male	USA	Phoenix	Q2
38333	Male	UK	Manchester	Q1
35963	Male	Australia	Adelaide	Q1
...	...	...	...	...
40747	Female	France	Nice	Q4
36799	Other	Germany	Cologne	Q3
41305	Female	UK	Manchester	Q2
15716	Male	Canada	Montreal	Q4
35801	Female	USA	Los Angeles	Q4

12588 rows × 4 columns

In [7]:
```
num_cols = Xtrain.select_dtypes(exclude='object').columns

low_cardinal_cols = ['Gender', 'Signup_Quarter', 'Country']
mid_cardinal_cols = ['City']
```

In [8]:
```
# Inspect cardinality of mid-cardinal columns
[X[col].value_counts().values for col in mid_cardinal_cols]
```

Out[8]:
```
[array([1297, 1253, 1225, 1220, 1212,  578,  562,  526,  519,  494,  451,
         449,  426,  414,  403,  371,  366,  343,  334,  325,  315,  313,
         310,  307,  300,  299,  290,  288,  287,  271,  267,  256,  248,
         245,  238,  208,  203,  198,  189,  183])]
```

The decay in city entries head counts drops after 5 most city present city, however the drop is not very significant to club as a seperate 'other' category.

In [ ]:
```
# Low cardinality: One-Hot Encoding
ohe = OneHotEncoder(sparse_output=False,
                    drop='first',
                    dtype=np.uint8).set_output(transform='pandas')
ohe.fit(Xtrain[low_cardinal_cols])

Xtrain_lowcard_ = ohe.transform(Xtrain[low_cardinal_cols])
Xtrain = pd.concat([Xtrain,
                    Xtrain_lowcard_], axis=1).drop(low_cardinal_cols, axis=1)
```

City acts as a subcategory within Country. One-hot encoding City would create a dimensionality problem and multicollinearity with Country.

From EDA, `City` shows little-to-no association with the target variable (`Churned`), ideally it can drop, though **frequency encoding** is used instead.

In [10]:
```
# Mid cardinality: Frequency Encoding  (via Series.map)
freq = Xtrain['City'].value_counts(normalize=True)

Xtrain['City_FE'] = Xtrain['City'].map(freq)
Xtrain = Xtrain.drop(columns='City')

Xtrain.dtypes.value_counts()
```

Out[10]:
```
float64    21
uint8      12
Name: count, dtype: int64
```

In [11]:
```
lr_cc = LogisticRegression()
lr_cc.fit(Xtrain, ytrain)
```

Out[11]:

▾ LogisticRegression

```
LogisticRegression()
```

Evaluate on Test Set

Since `df_dropped` contains no missing values, `Xrest` is already a clean complete-case holdout — no filtering needed.

In [12]:
```
# One-Hot Encoding — apply transform learned on Xtrain
Xrest_lowcard_ = ohe.transform(Xrest[low_cardinal_cols])
Xrest = pd.concat([Xrest, Xrest_lowcard_], axis=1).drop(low_cardinal_cols, axis=1)

# Frequency Encoding — map using freq learned on Xtrain
Xrest['City_FE'] = Xrest['City'].map(freq)
Xrest = Xrest.drop(columns='City')

score_cc = lr_cc.score(Xrest, yrest)
print(f'Model 1 (Complete-Case) — Test Accuracy: {score_cc:.4f}')
```

```
Model 1 (Complete-Case) — Test Accuracy: 0.7690
```

Model 2 — Imputed Training

All rows are retained; missing numeric values are filled via mean imputation on the training set. The model is then evaluated on two test subsets:

- **Full test set** — all holdout rows, including imputed ones
- **Clean test set only** — holdout rows with no original missing values; this is the apples-to-apples comparison against Model 1

> `missing_flag` is used **only** for stratified splitting and test-set filtering. It is never seen by the model as a feature.

In [ ]:
```
X = df.drop('Churned', axis=1)
y = df['Churned']
```

```
missing_flag = df.isna().any(axis=1).astype(int)

# Combine missingness pattern and target into a single stratification key
# so both proportions are preserved across train/test
stratify_key = missing_flag.astype(str) + '_' + y.astype(str)

Xtrain, Xrest, ytrain, yrest, mflag_train, mflag_rest = train_test_split(
    X, y, missing_flag,
    train_size=0.7,
    stratify=stratify_key,
    random_state=42
)

print('All rows, no drop:')
print(f'Train size: {len(Xtrain):,}  (rows with imputed values: {mflag_train.sum():,})')
print(f'Test size:  {len(Xrest):,}  (rows with imputed values: {mflag_rest.sum():,})')
```

```
All rows, no drop:
Train size: 35,000  (rows with imputed values: 22,412)
Test size:  15,000  (rows with imputed values: 9,605)
```

Impute Missing Values

The imputer is fit **only on the training set** to prevent data leakage.

In [14]:
```
num_cols = Xtrain.select_dtypes(exclude=object).columns
mean_imputer = SimpleImputer().set_output(transform='pandas')
mean_imputer.fit(Xtrain[num_cols])

Xtrain_missing_fix = mean_imputer.transform(Xtrain[num_cols])
Xtrain = Xtrain.combine_first(Xtrain_missing_fix)
```

Encode Categorical Features

In [15]:
```
low_cardinal_cols = ['Gender', 'Signup_Quarter', 'Country']
mid_cardinal_cols = ['City']
```

Low Cardinality — One-Hot Encoding

In [ ]:
```
ohe = OneHotEncoder(
    sparse_output=False, drop='first',
    dtype=np.uint8,
    handle_unknown='ignore').set_output(transform='pandas')
ohe.fit(Xtrain[low_cardinal_cols])
```

Out[ ]:

OneHotEncoder

```
OneHotEncoder(drop='first', dtype=<class 'numpy.uint8'>,
              handle_unknown='ignore', sparse_output=False)
```

In [17]:
```
Xtrain_lowcard_ = ohe.transform(Xtrain[low_cardinal_cols])
Xtrain = pd.concat([Xtrain, Xtrain_lowcard_], axis=1).drop(low_cardinal_cols, axis=1)
```

Mid/Moderate Cardinality — Frequency Encoding

Same semantic output as Model 1, but using `DataFrame.replace` instead of `Series.map` — intentionally.

In [18]:
```
freq_map = Xtrain[mid_cardinal_cols].value_counts(normalize=True)
Xtrain[mid_cardinal_cols] = Xtrain[mid_cardinal_cols].replace(freq_map)

Xtrain.dtypes.value_counts()
```

Out[18]:
```
float64    21
uint8      12
Name: count, dtype: int64
```

Train Model

In [19]:
```
lr_imp = LogisticRegression()
lr_imp.fit(Xtrain, ytrain)
```

Out[19]:
```
LogisticRegression()
```

Evaluate

Apply the same preprocessing pipeline (using parameters fit on training data) to the test set before scoring.

In [20]:
```
# Impute — apply training-set statistics to test rows
Xrest_missing_fix = mean_imputer.transform(Xrest[num_cols])
Xrest = Xrest.combine_first(Xrest_missing_fix)

# Low cardinal features
Xrest_lowcard_ = ohe.transform(Xrest[low_cardinal_cols])
Xrest = pd.concat([Xrest, Xrest_lowcard_], axis=1).drop(columns=low_cardinal_cols)

# Mid cardinal features  (via .replace — consistent with training encoding)
Xrest[mid_cardinal_cols] = Xrest[mid_cardinal_cols].replace(freq_map)
```

In [ ]:
```
# Full test set: includes rows that had missing values (now imputed)
score_imp_all = lr_imp.score(Xrest, yrest)
print(f'Model 2 (Imputed) — Full test set accuracy: {score_imp_all:.4f}')
```

```
Model 2 (Imputed) — Full test set accuracy:         0.7695
```

In [ ]:
```
# ground truth test rows only: rows that had no missing values in the original data
# This is the fair comparison against Model 1
clean_mask = mflag_rest == 0
score_imp_clean = lr_imp.score(Xrest[clean_mask], yrest[clean_mask])
print(f'Model 2 (Imputed) — Ground-truth test set accuracy: {score_imp_clean:.4f}')
```

```
Model 2 (Imputed) — Ground-truth test set accuracy:  0.7772
```

Results — Side-by-Side Comparison

The **primary comparison** is Model 1 vs. Model 2 on the clean test set. Both models are scored on identical rows, so any difference isolates the effect of training with imputed data.

In [23]:
```
comparison = pd.DataFrame({
    'Model': [
        'Model 1 — Complete-Case Training',
        'Model 2 — Imputed Training  [primary comparison]',
        'Model 2 — Imputed Training  [reference]',
    ],
    'Test Set': [
        'Ground truth rows only',
        'Ground truth rows only',
        'All rows (incl. imputed)',
    ],
    'Accuracy': [score_cc, score_imp_clean, score_imp_all],
})
comparison.set_index('Model')
```

Out[23]:

Model	Test Set	Accuracy
Model 1 — Complete-Case Training	Ground truth rows only	0.769045
Model 2 — Imputed Training [primary comparison]	Ground truth rows only	0.777201
Model 2 — Imputed Training [reference]	All rows (incl. imputed)	0.769467

Interpreting the Result

Outcome	Interpretation
`Model 2 (ground-truth) ≈ Model 1`	Imputation neither helps nor hurts on known-value records — safe to proceed
`Model 2 (ground-truth) > Model 1`	Imputation added useful signal; note the sample-size confound (more training rows)
`Model 2 (ground-truth) < Model 1`	Imputed values introduced noise that degraded the decision boundary — imputation method may not be justified for this dataset

Note: The notebook demonstrate accuracy metrics and logistic regression for comparision, use metric and algorithm of your choice.